1 Evaluation Appendix

All case study source code, specifications, and grading infrastructure are available in a single public repository (`bsc-apm-case-study-task`).[1]

1.1 Case Study Repositories

The case study uses two repositories. The task repository (`bsc-apm-case-study-task`) is openly accessible and holds task specifications, grading infrastructure, and participant guides, organized into three top-level directories: `task/` (Product Requirements Document, default prompt, scoring rubric), `grader/` (black-box test suite, data processing pipeline, chat parsers, SSO mock fixtures), and `participant-guides/` (LaTeX participant guide for GitHub Copilot). The dataset repository (`bsc-apm-case-study-data`) holds all submitted participant data: code patches, sanitized chat transcripts, and anonymized session metadata. Access to the dataset repository requires explicit authorization and compliance with data usage agreements, enabling peer review and reproducibility while maintaining participant privacy.

1.2 Baseline Reproducibility

All tests run against fixed baseline commits:

- `eclass-mcp-server` at commit `8b4e39c` (main branch).
- `openeclass` at commit `e8b3329` (Docker image `openeclass:test`).

The grading infrastructure applies participant patches against these exact versions. A submission with no patch applied receives the baseline score of 12/100, representing only the existing Python functionality. Environment setup, Docker configuration, and test data seeding are documented in `grader/SETUP.md`.

1.3 Grader Environment Replication

Grading runs against a Docker-based replica of the openeclass platform rather than against the live University of Athens instance that participants develop against. The replica pins openeclass to the baseline commit `e8b3329`, seeds deterministic fixture data for courses, announcements, and users, and handles authentication through an SSO mock standing in for the live SAML provider. Together these make every grading run reproducible: the same patch produces the same score independent of live-instance availability or content drift.

The replica and the live instance are not bit-identical. Divergence modes that can in principle affect grading outcomes (timezone handling of announcement dates, locale-sensitive parsing of Greek date strings, session-cookie lifecycle and scoping, and cross-origin behavior between the MCP server and the SSO mock) are discussed as a threat to construct validity in the Discussion. Replica-construction details live in the grader `SETUP.md` and `PIPELINE.md` documents.

[1] Repository available at: https://github.com/sdi2200262/bsc-apm-case-study-task

1.4 Grading Pipeline

The grading pipeline has six stages with manifest-based checkpointing:

1. Validation: verifies zip structure and naming format.
2. Grading: runs the black-box test suite against Python and C implementations.
3. Parsing: extracts interaction metrics from chat exports and applies the encryption described in Section 1.6.
4. Combination: merges grading results with interaction metrics into unified result files.
5. Dataset Aggregation: appends processed data to platform-level CSV datasets.
6. Integrity Validation: verifies dataset consistency against manifest checksums.

Manifests are stored per-platform, recording checksums and timestamps at each stage. Pipeline documentation is in `grader/PIPELINE.md`.

1.5 Submission Workflow and Format

Each participant completes the task twice in isolated sessions, each session time-boxed to three hours. For each session, the participant creates a dedicated Git branch from the clean `main` baseline (for example, `s1_apm`), works on the task using the assigned framework, then generates a solution patch as the diff between `main` and the session branch. After exporting chat logs and saving the patch, the session branch is deleted before the second session begins.

1.5.1 Submission Format

Submissions follow a fixed naming convention for automated processing:

`{PID}_{S1|S2}_{platform}_{tool}_submission.zip`

The fields are: participant ID (for example, `P001`), session number, platform identifier (`copilot`), and framework identifier (`apm` or `spec-kit`). Each zip contains two items:

1. A patch file (`_solution.patch`) generated via `git diff main <branch>`.
2. A logs folder (`_logs/chats/`) containing exported Copilot conversations in JSON format.

The per-session submission is uploaded alongside the post-session questionnaire response through a single web form. Full instructions for environment setup, chat export, and submission packaging are provided in the participant guide (`participant-guide-copilot.tex`).

1.6 Data Privacy and Sanitization

All chat exports are anonymized and encrypted during the parsing stage of the grading pipeline.

Each conversation is assigned a unique chat ID computed as the first 16 hexadecimal characters of a SHA256 hash over concatenated user messages. This ID is computed before encryption and preserved in plaintext alongside the following metadata: distinct model IDs, total request count, sanitization timestamp, and original payload size. The metadata enables dataset organization and integrity validation without requiring decryption.

Conversation content is encrypted using an HMAC-SHA256 XOR cipher. A random salt seeds keystream expansion; the serialized chat payload is XOR-encrypted against this keystream and stored in the format `ENC:<salt>:<ciphertext>`, both Base64-encoded. Decryption with the shared secret produces output bitwise-identical to the original.

1.7 Copilot Procurement Model

GitHub Copilot is sold as a three-part tariff: a per-seat monthly access price, an included allowance of premium requests per user per month, and a marginal charge of $0.04 for each premium request beyond the allowance.[2] Allowance sizes as of May 2026 are: Student (300 premium requests per month, free through the GitHub Education Pack), Pro (300), Pro+ (1,500), Business (300 per seat), and Enterprise (1,000 per seat).

Each supported model is associated with a **premium-request multiplier** that determines how many premium requests a single model interaction consumes. A 1× model consumes one premium request per interaction; reasoning and fast-mode models apply larger multipliers (common values are 3×, 5×, and 30×). The published multiplier table is periodically updated, and promotional multipliers are time-limited and subject to vendor change.[3]

All case-study sessions use GPT-5.4, a 1× premium-request model available on every plan tier relevant to the participant pool. Holding the model fixed at a uniform 1× multiplier eliminates multiplier-induced variance in the economic metric across frameworks and across the two sessions of each participant.

The Copilot cloud-agent feature invokes a second consumption stream alongside premium requests (GitHub Actions minutes), but neither APM nor Spec-kit invokes cloud agents in the testbed task, so the dual-consumption mechanism is out of scope for the case study.

1.8 Framework Phase Detection

The secondary economic metric and the Analysis Plan both rely on a planning-versus-implementation phase split for each framework session. Because the two frameworks express phases differently, detection uses framework-specific rules applied at parse time.

[2] Tiered plans and allowances documented at: https://docs.github.com/en/copilot/get-started/plans; premium-request billing and overage rates documented at: https://docs.github.com/en/billing/concepts/product-billing/github-copilot-premium-requests

[3] Supported models and plan availability documented at: https://docs.github.com/en/copilot/reference/ai-models/supported-models

For **APM**, the phase boundary coincides with conversation identity: the Planner conversation counts as planning phase, and the Manager and Worker conversations count as implementation phase. Conversation identities are preserved in the chat-export filenames and in the exported metadata, so detection is a lookup rather than a heuristic.

For **Spec-kit**, the phase boundary is the `/speckit.implement` slash-command invocation. All requests from the first user message up to and including the last pre-implement command count as planning phase; the pre-implement command set is `/speckit.constitution`, `/speckit.specify`, `/speckit.plan`, `/speckit.tasks`, `/speckit.clarify`, `/speckit.analyze`, and `/speckit.checklist`. All requests from `/speckit.implement` onward count as implementation phase.

Detection is applied independently per participant submission, and the phase-broken counts are aggregated at the cohort level in the results analysis.

1.9 Test Suite Composition

The test suite has three categories totaling 100 points:

- Python Baseline (12 points): 11 test cases covering build, protocol compliance, login, logout, courses, and authentication status.
- Python Features (36 points): 12 test cases split evenly between the two announcement tools (18 points each), covering error handling, date filtering, output format, and source aggregation.
- C Replica (52 points): 28 test cases covering compilation, Valgrind memory safety, protocol compliance, and all six tools (login, logout, courses, authstatus, and both announcement tools).

Point allocations and test definitions are in `grader/src/scorer.py`; the scoring rubric is in `task/SCORING.md`.